AssemblyFlow: Pipelined Execution Analyzer

Subject Micro Project Report: Formal Languages & Automata Theory (FLAT)

Student Name:	Ashik Ameen	**Subject:**	Formal Languages & Automata Theory (FLAT)
Project:	AssemblyFlow	**Academic Term:**	October 2026
Live Demo:	https://micro-project-a338.onrender.com/	**GitHub Repo:**	https://github.com/Ashik9500/Micro_project.git

1. Executive Summary & Problem Statement

In the design of modern compiler pipelines and assembly tools, lexical scanning and grammar validation form the indispensable front-end. In the integrated **AssemblyFlow** micro project, the **Formal Languages and Automata Theory (FLAT)** module serves as **Stage 1**. It normalizes unstructured assembly source text into categorized token streams, evaluates regular token structures using a formal Deterministic Finite Automaton (DFA), enforces strict Context-Free Grammar (CFG) production rules, and implements a safe execution-halting invariant to protect downstream dependency and simulation stages.

2. Theoretical Foundations in FLAT

2.1 Language Classification under the Chomsky Hierarchy

AssemblyFlow models assembly language statements across Chomsky's hierarchy:
• **Type 3 (Regular Languages):** Register tokens (R0–R15), immediate integers, and commas are regular patterns recognized by Finite State Automata.
• **Type 2 (Context-Free Languages):** Multi-operand instruction statements with ordered operands and strict comma delimiters are validated via CFG production rules.

2.2 Formal 5-Tuple Register Token DFA: M = (Q, Σ, δ, q0, F)

The language of hardware register identifiers *L = {R0, R1, ..., R15}* (case-insensitive) is recognized by the formal 5-tuple deterministic finite automaton **M = (Q, Σ, δ, q0, F)**, where:
• **Q:** {q0, q1, q2, q3, q4, q5, q_trap}
• **Σ:** {'R', 'r', '0', '1', '2'–'9', other}
• **q0:** Initial start state.
• **F:** {q2, q3, q4, q5} (Accepting states for R0, R1, R2–R9, R10–R15 respectively).
• **q_trap:** Dead state absorbing all invalid prefixes, out-of-range indices (e.g. R16), and trailing characters.

Source State	Symbol Class	Target State	Automaton Meaning / Description
q0	'R', 'r'	q1	Valid register prefix recognized
q0	other	q_trap	Invalid prefix violation (e.g. 'X', '1')
q1	'0'	q2 (Accepting)	Matches register R0
q1	'1'	q3 (Accepting)	Matches R1 or initiates tens-digit for R10-R15
q1	'2'–'9'	q4 (Accepting)	Matches single-digit registers R2 through R9
q3	'0'–'5'	q5 (Accepting)	Matches two-digit registers R10 through R15
q3	'6'–'9'	q_trap	R16–R19 exceeds hardware boundary (R0–R15)
q4, q5	any	q_trap	Disallows multi-digit registers >= 20 or trailing characters
q_trap	any	q_trap	Dead state loops unconditionally on all input

2.3 Context-Free Grammar Production Rules (EBNF)

AssemblyFlow enforces the following formal instruction production rules:
• LOAD destinationRegister, integer (Immediate Load)
• MOV destinationRegister, sourceRegister (Register Transfer)
• ADD destinationRegister, sourceRegister1, sourceRegister2 (ALU Addition)
• SUB destinationRegister, sourceRegister1, sourceRegister2 (ALU Subtraction)

3. System Implementation & Architecture in AssemblyFlow

The FLAT engine is implemented in TypeScript within assemblyflow/backend/src/services/flat/:
• **tokenizer.ts:** Scans lines in O(N) linear time, identifying opcodes, registers, integer literals, commas, and comments while logging 1-indexed column coordinates.
• **dfa.ts:** Simulates the formal 5-tuple automaton, producing comprehensive transition step diagnostics.
• **validator.ts:** Checks operand counts, comma positions, and register bounds, generating clean InstructionAST nodes.
• **Stage-Level Safety Halting:** If any syntax or lexical error is detected, the server responds with HTTP 400 and immediately terminates execution before Stages 2 (DAA) and 3 (COA) can execute.

4. Experimental Test Cases & Verification

Test Case	Input Assembly Statement	FLAT Evaluation	Pipeline Result
Valid Load	LOAD R1, 100	Accepted: Valid opcode, R1 in range, valid integer	Passes to Stage 2 (DAA)
Valid ALU	ADD R3, R1, R2	Accepted: Valid 3-operand register syntax	Passes to Stage 2 (DAA)
Out-of-Bounds Reg	MOV R16, R1	Rejected: R16 transitions to q_trap (Exceeds R15)	HALT: HTTP 400 (FLAT Error)
Missing Operand	ADD R1, R2	Rejected: Expected 3 operands, found 2	HALT: HTTP 400 (FLAT Error)
Unknown Opcode	MUL R1, R2	Rejected: Opcode 'MUL' not in supported set	HALT: HTTP 400 (FLAT Error)

5. Asymptotic Complexity & Conclusions

Time Complexity: Line scanning and tokenization execute in O(L) time per line, yielding O(N) total time for a program of N characters. DFA transition verification executes in O(1) time per token (string length ≤ 4). Total parsing complexity is strictly optimal linear time O(N).
Learning Outcomes: Practical software engineering demonstration of finite automata, formal language grammars, lexical token classification, and compiler front-end error recovery.